End-to-End CI/CD Pipeline for a Calculator REST API

CSE 816 Software Production Engineering - Mini-Project Report

Nikunj Mahajan (IMT2023068)

GitHub repository	https://github.com/nikunj169/calculator-devops
Docker Hub image	https://hub.docker.com/r/niki9937/calculator-devops (public, tags *:latest*, *:<build number>*)
Stack	Java 21, Spring Boot 3.5 (Web + Actuator), Maven, JUnit 5 / MockMvc, Git/GitHub, Jenkins LTS (declarative pipeline), Docker (multi-stage), Docker Compose, Nginx

1. Overview

The project is a small Calculator REST API wrapped in a fully automated delivery pipeline. A change merged into **main** on GitHub is detected by Jenkins, compiled, tested, packaged into a Docker image tagged with the Jenkins build number, pushed to Docker Hub and rolled out with Docker Compose behind an Nginx reverse proxy. The pipeline then checks the deployment's health, runs a smoke test, and emails the build result. No step needs manual intervention; build #1 was triggered by Poll SCM after the pull requests were merged and finished green in about four minutes.

2. Pipeline architecture

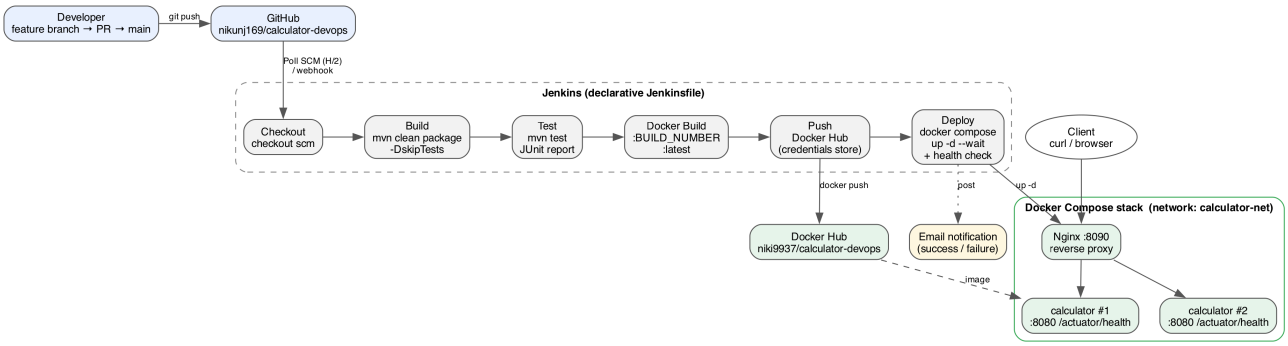


Figure 1 - Flow from a Git push through the six Jenkins stages to Docker Hub and the Compose deployment.

3. Application

Endpoints: `GET /api/v1/{add,subtract,multiply,divide}?a=&b=` return `{"operation":"add","a":2,"b":3,"result":5}`; `/actuator/health` reports UP/DOWN. Arithmetic lives in a plain *CalculatorService*; the controller only validates input and wraps results. A *@RestControllerAdvice* maps missing parameters, non-numeric values, NaN/Infinity and division by zero to HTTP 400 with a structured body (timestamp, status, error, message, path). Whole-number results are rendered as integers so the JSON matches the specification (5, not 5.0).

Tests (30, all passing): parameterised JUnit 5 unit tests for every operation, including divide-by-zero and 0/0; a *@SpringBootTest* + MockMvc integration suite that calls each real endpoint, every error path and the Actuator health/info endpoints. `mvn test` writes Surefire JUnit XML, which Jenkins publishes.

4. Pipeline stages

Stage	What it does
Trigger	`githubPush()` for a GitHub webhook, and `pollSCM('H/2 * * * *')` as the working trigger, since this Jenkins runs on localhost and GitHub cannot reach it. `disableConcurrentBuilds` and a 30-minute timeout keep runs from overlapping.
Checkout	`checkout scm` pulls the commit that triggered the build and logs it.
Build	`mvn -B clean package -DskipTests` produces *target/calculator.jar*.
Test	`mvn -B test`; a non-zero exit fails the stage and the build. `junit 'target/surefire-reports/*.xml'` runs in *post { always }*, so results are published even on failure.
Docker Build	Builds the multi-stage image and tags it *niki9937/calculator-devops:<BUILD_NUMBER>* and *:latest*.

Stage	What it does
Push	Binds the Jenkins credential *dockerhub-credentials* via `credentials()` (masked in logs), logs in with `--password-stdin` using a throwaway `DOCKER_CONFIG`, pushes both tags, logs out and deletes the temporary config.
Deploy	`docker compose up -d --remove-orphans --wait` with `IMAGE_TAG=BUILD_NUMBER`, then polls */actuator/health* through Nginx and runs a smoke request; on failure it prints container logs and fails the build.
Post	`emailext` sends an HTML success mail or a failure mail with the console log attached; dangling images are pruned.

Idempotency: the Compose project name is fixed (*calculator-devops*), so every run updates the same stack. `up -d` only recreates containers whose image or config changed, and a re-run of the same commit leaves the running deployment alone. **Secrets:** no credentials live in the repository; *.gitignore* excludes *target/*, IDE files, *.env* and key files.

5. Containerisation and deployment

Dockerfile (multi-stage): stage 1 (*maven:3.9-eclipse-temurin-21*) copies *pom.xml* first and runs `dependency:go-offline`, so the dependency layer stays cached until the POM changes, then copies *src/* and packages the JAR. Stage 2 (*eclipse-temurin:21-jre-alpine*, about 95 MB) copies only the JAR, creates and switches to a non-root *app* user, declares `EXPOSE 8080` and a `HEALTHCHECK`. A *.dockerignore* limits the build context to *pom.xml* and *src/*. `docker run -p 8085:8080 niki9937/calculator-devops` is all it takes to run it.

docker-compose.yml: two *calculator* replicas (`deploy.replicas: 2`) and an *nginx:1.27-alpine* reverse proxy published on port 8090, on a dedicated bridge network *calculator-net*. Both services have health checks (the API checks */actuator/health* for *"status":"UP"*), `restart: unless-stopped`, and environment variables (*SERVER_PORT*, *JAVA_TOOL_OPTIONS*, *TZ*). Nginx starts only after the replicas report *service_healthy*. It resolves the service name through Docker DNS on every request, which spreads load across replicas (the *X-Upstream-Addr* header alternates) and follows new container IPs after a redeploy. One `docker compose up -d` starts the whole stack.

6. Git workflow

The application was built in small commits on *main* (skeleton, service + unit tests, controller + validation, integration tests). Containerisation, the Jenkinsfile and the README each went through their own feature branch and pull request (#1-#3), merged with merge commits so the branch history stays visible. PR #4 (deployed build number on */actuator/info*) is the change for the live demo: merging it makes Jenkins build, push and deploy the next build automatically. Pushing this report to main already did the same thing: build #2 was triggered by Poll SCM and redeployed only the API replicas, leaving Nginx running.

7. Challenges faced

Problem	Resolution
***eclipse-temurin:17-jre-alpine* has no arm64 image, so the build failed on Apple Silicon (*no match for platform in manifest*).**	Moved to Java 21 LTS, whose Alpine JRE is multi-arch and matches the JDK Jenkins runs on.
The Nginx container stayed *unhealthy*: *localhost* resolved to ::1 inside Alpine, but Nginx listens on IPv4 only.	Health checks use 127.0.0.1 explicitly.
Nginx caches upstream IPs at startup, so after Jenkins recreated the API containers it would proxy to dead addresses.	Used `resolver 127.0.0.11` with a variable in `proxy_pass`, which forces a DNS lookup per request.
The integration test class, at first named **IT*, was silently skipped by Surefire.	Renamed it to *CalculatorControllerIntegrationTest* so `mvn test` runs it.
Jenkins runs on localhost, so a GitHub webhook can't reach it.	Kept `githubPush()` for a public Jenkins and added Poll SCM every 2 minutes.
`docker login/logout` **in the pipeline would overwrite or wipe the host's Docker Desktop keychain login.**	The Push stage uses a temporary `DOCKER_CONFIG` with no credential helper and deletes it afterwards.
Jenkins started by launchd has no Homebrew PATH (*mvn*, *docker*).	Prepended */opt/homebrew/bin* in the pipeline's `environment`.

Problem	Resolution
No SMTP account was configured in Jenkins.	Pointed Email Extension at a local Mailpit SMTP server (localhost:1025) for the demo. Real delivery only needs Gmail SMTP (smtp.gmail.com:465, SSL, app password) configured in Jenkins; the Jenkinsfile stays the same.

Known limitation: the image is built on the Jenkins host, so it is arm64-only. A `docker buildx --platform linux/amd64,linux/arm64` build would make it multi-arch.

Appendix - Screenshots

A. Jenkins pipeline

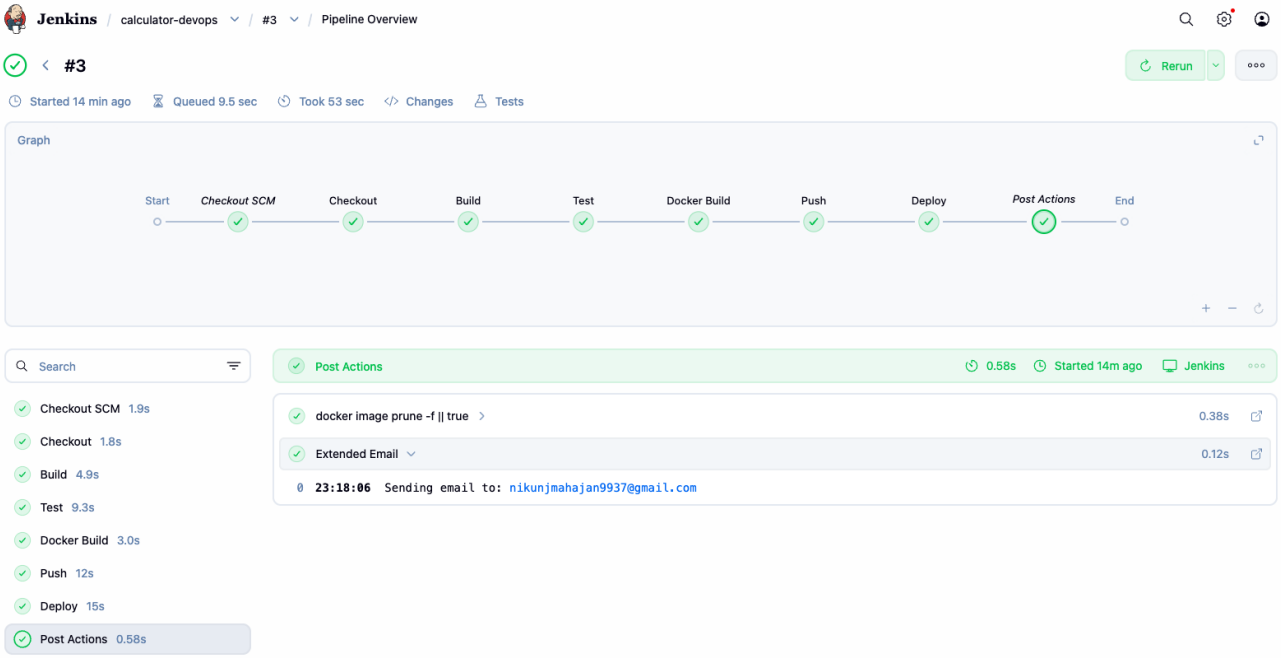


Figure 2 - Pipeline Overview of build #3: every stage (Checkout, Build, Test, Docker Build, Push, Deploy, Post Actions) is green; the run took 53 s and ended by emailing the result.

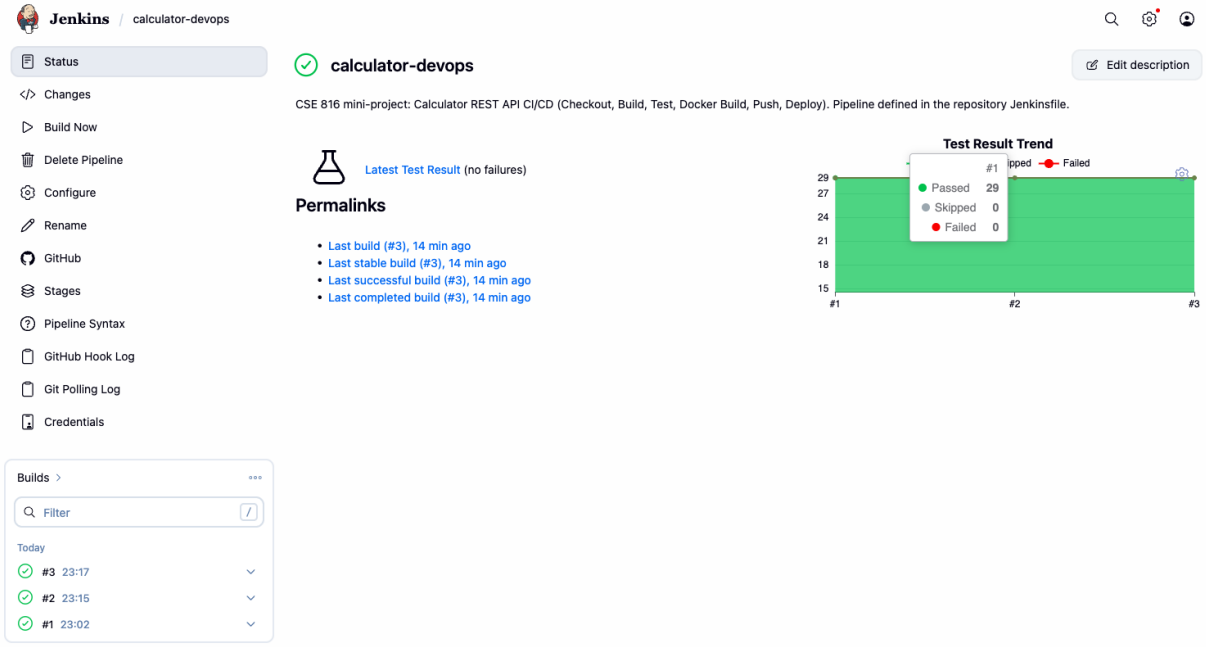


Figure 3 - The *calculator-devops* job: builds #1-#3 all successful (each one started by an SCM change), with the test-result trend at 29 passing tests.

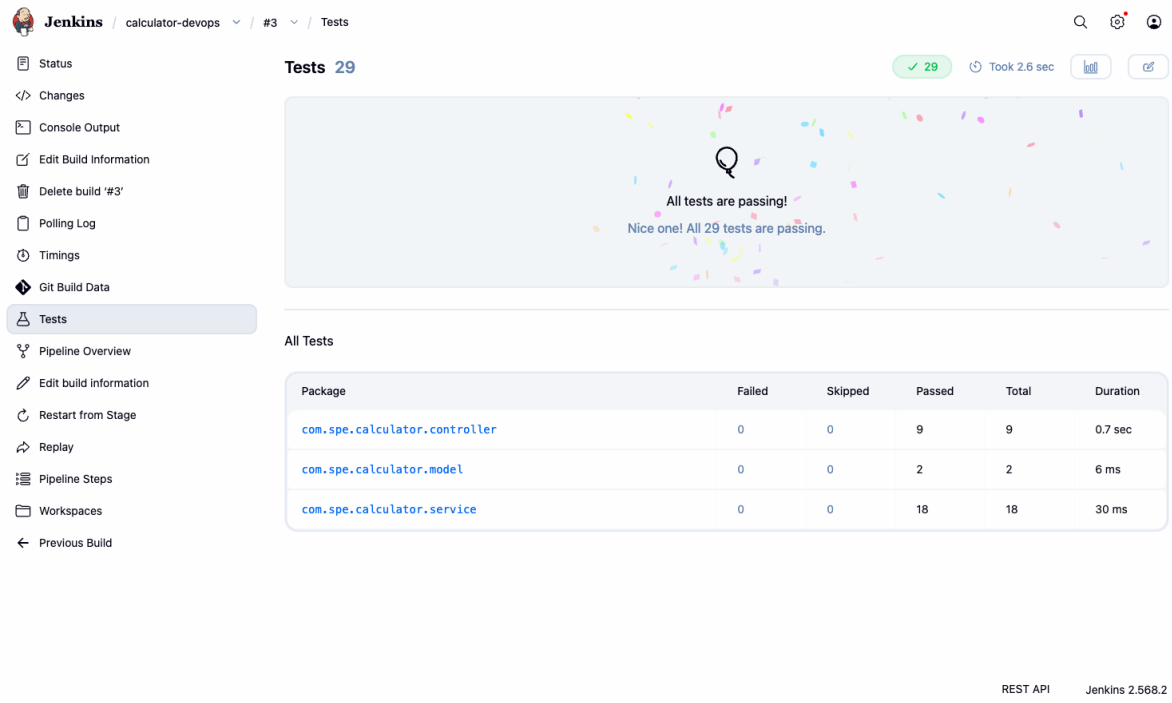


Figure 4 - JUnit results published by the Test stage: 29/29 passing across the controller (MockMvc), model and service packages.

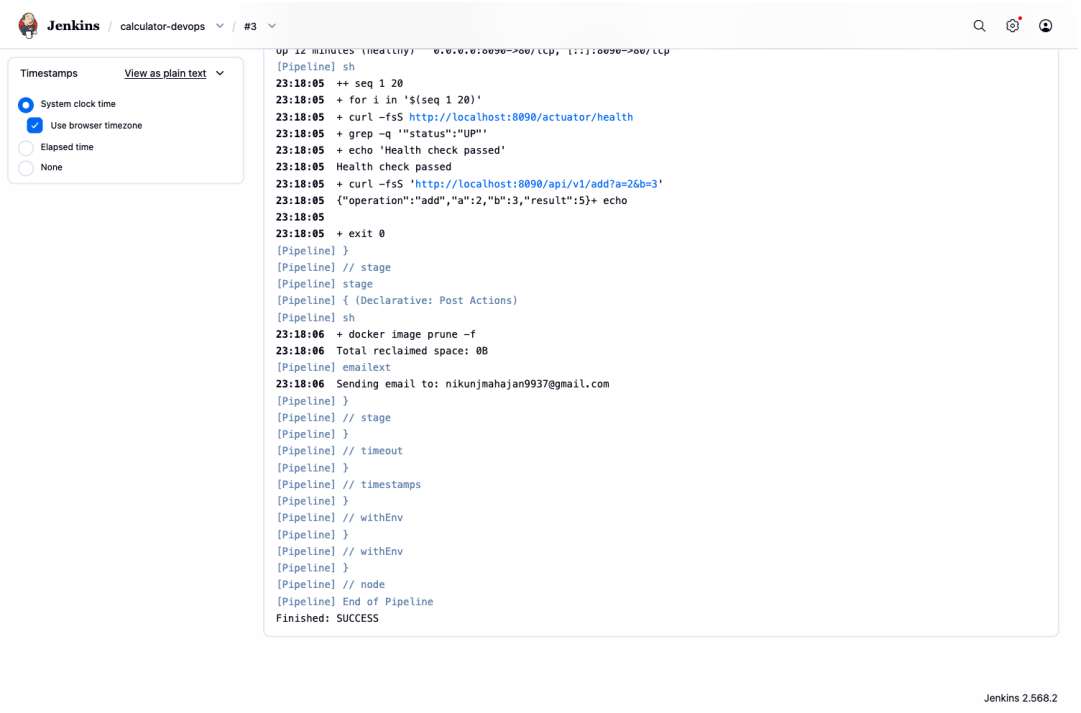


Figure 5 - End of the console log for build #3: health check and smoke test through Nginx, email sent, **Finished: SUCCESS**.

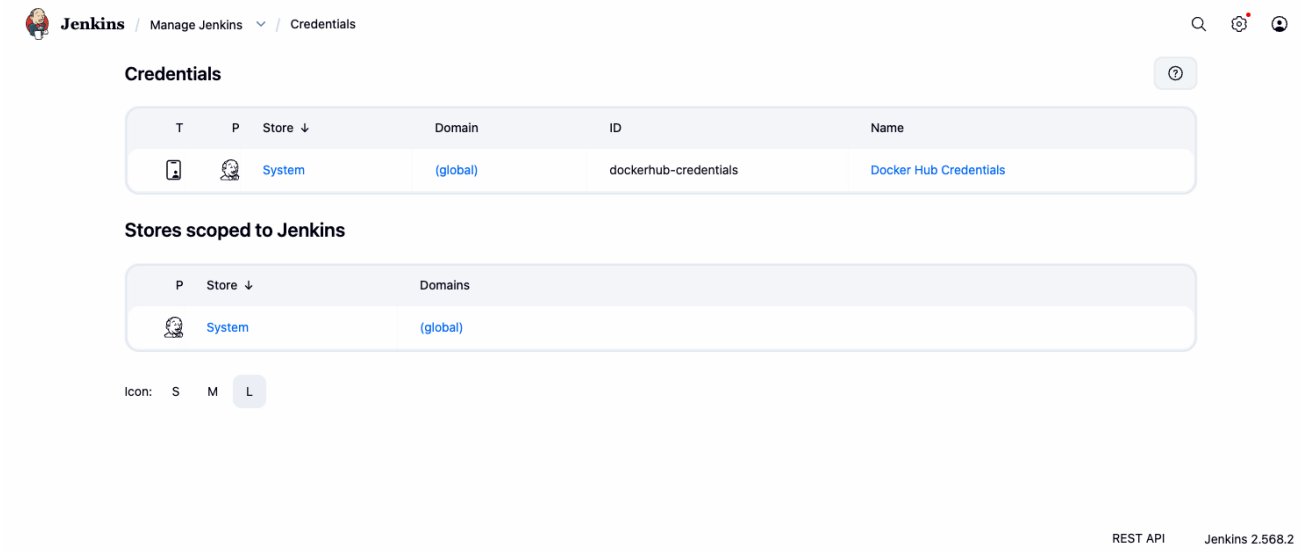


Figure 6 - The Docker Hub login is kept in the Jenkins credentials store (*dockerhub-credentials*), not in the repository.

B. Email notification

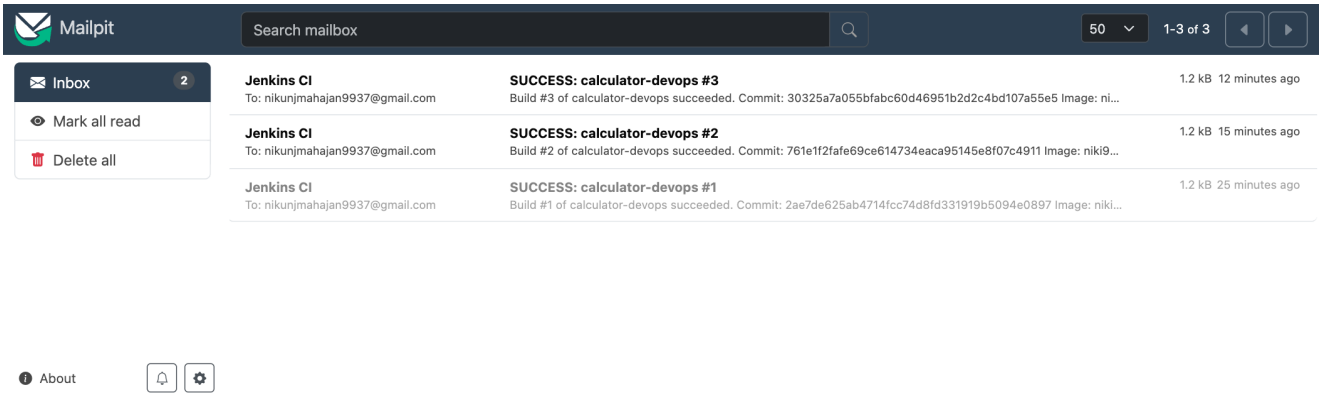


Figure 7 - One notification per build, all addressed to the student's email.

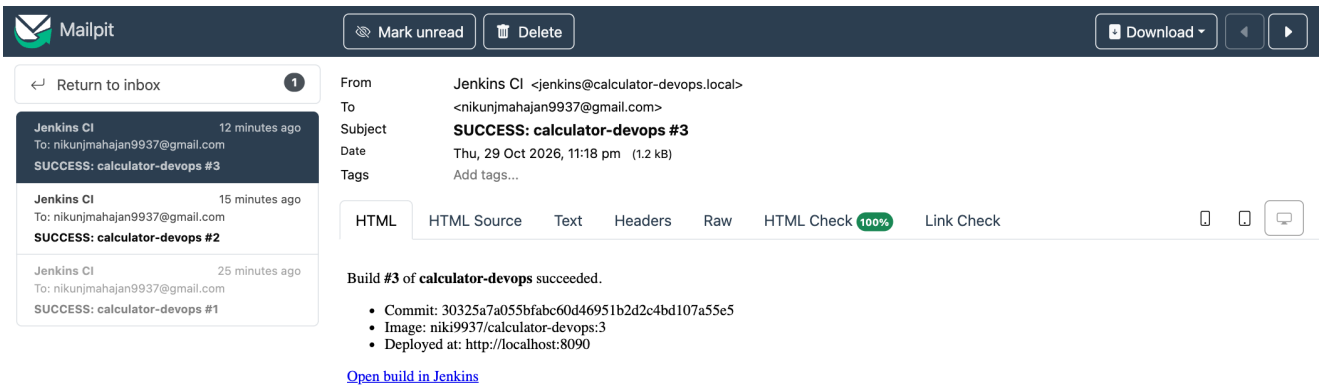


Figure 8 - The success email for build #3 with the commit, image tag and deployment URL.

C. Running deployment

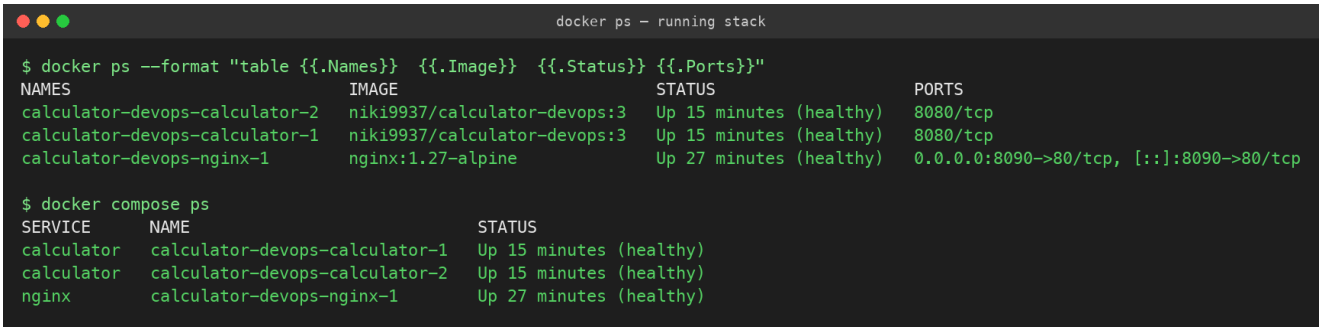


Figure 9 - `docker ps`: two API replicas on image tag *:3* and Nginx, all healthy. Nginx has been up longer than the API containers because a redeploy recreates only the containers whose image changed.



Figure 10 - Live responses through Nginx on :8090 in the browser: add, divide by zero (HTTP 400 error body) and */actuator/health*.

D. Docker Hub and GitHub

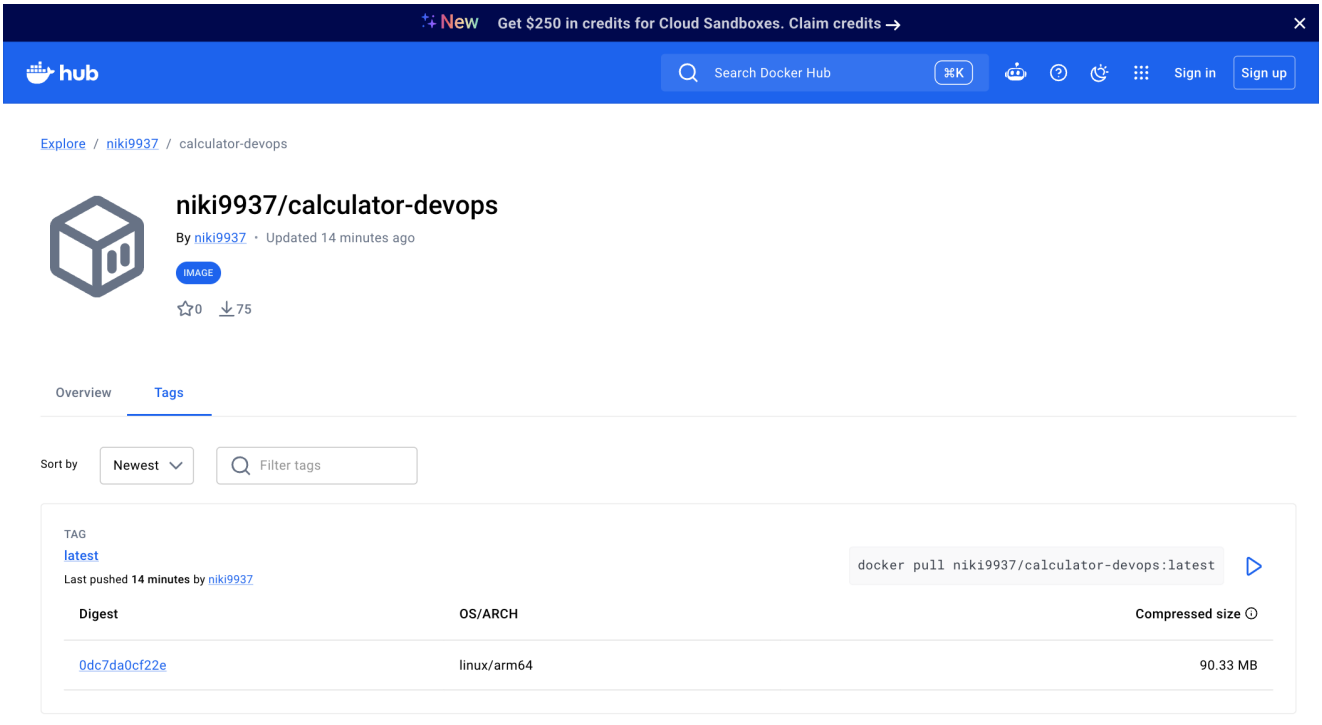


Figure 11 - Public Docker Hub repository *niki9937/calculator-devops* with the tags pushed by Jenkins.

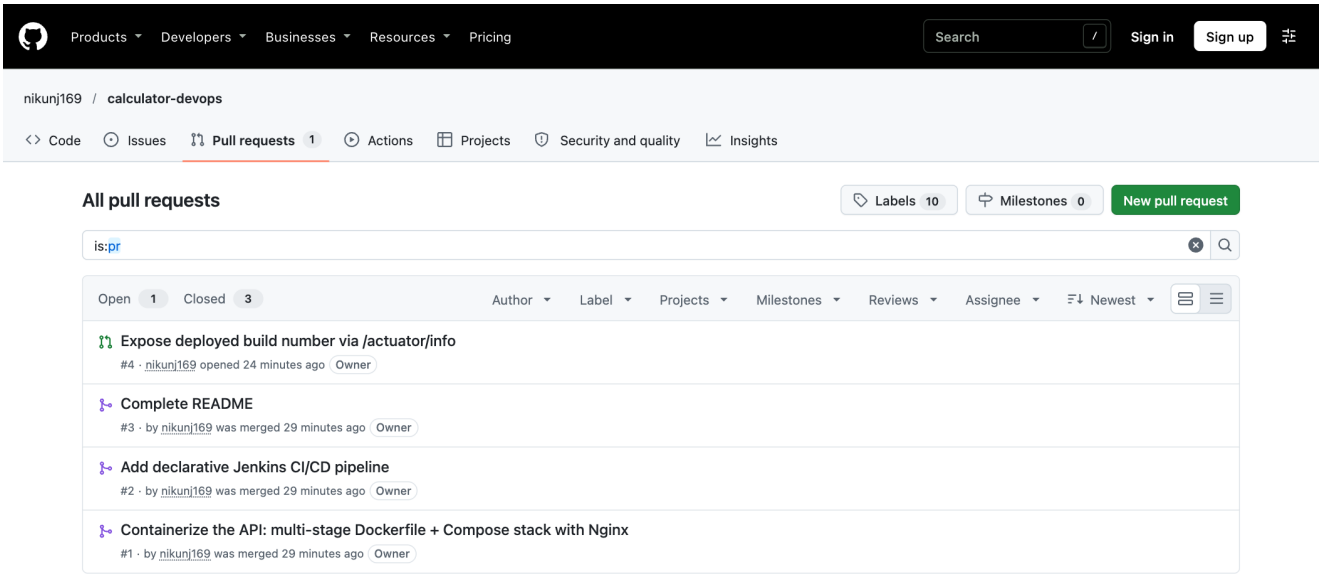


Figure 12 - Pull requests: feature branches merged through PRs #1-#3; PR #4 is kept for the live demo.

```
git log --graph --oneline --all

* 2ddab71 Expose deployed build number via /actuator/info
*   2ae7de6 Merge pull request #3 from nikunj169/docs/readme
|\
| * c6b2e83 Document build, test, run, API usage and the CI/CD pipeline in README
* |   7baa871 Merge pull request #2 from nikunj169/feature/jenkins-pipeline
|\ \
| * | c98f8db Add declarative Jenkins pipeline
* | | 0358e07 Merge pull request #1 from nikunj169/feature/containerization
|\| |
| |/
|/|
| * f0064b9 Add Docker Compose stack with Nginx reverse proxy and 2 API replicas
| * 5520240 Add multi-stage Dockerfile and .dockerignore
| * 08a020b Target Java 21 LTS
|/
* 09466cc Add MockMvc integration tests for all endpoints and error cases
* 4da9150 Add REST endpoints for add/subtract/multiply/divide with input validation
* 5569c73 Add CalculatorService with JUnit 5 unit tests for all operations
* 29474ab Initial Spring Boot project skeleton with Maven build and .gitignore
```

Figure 13 - Commit history showing the feature branches and merge commits.

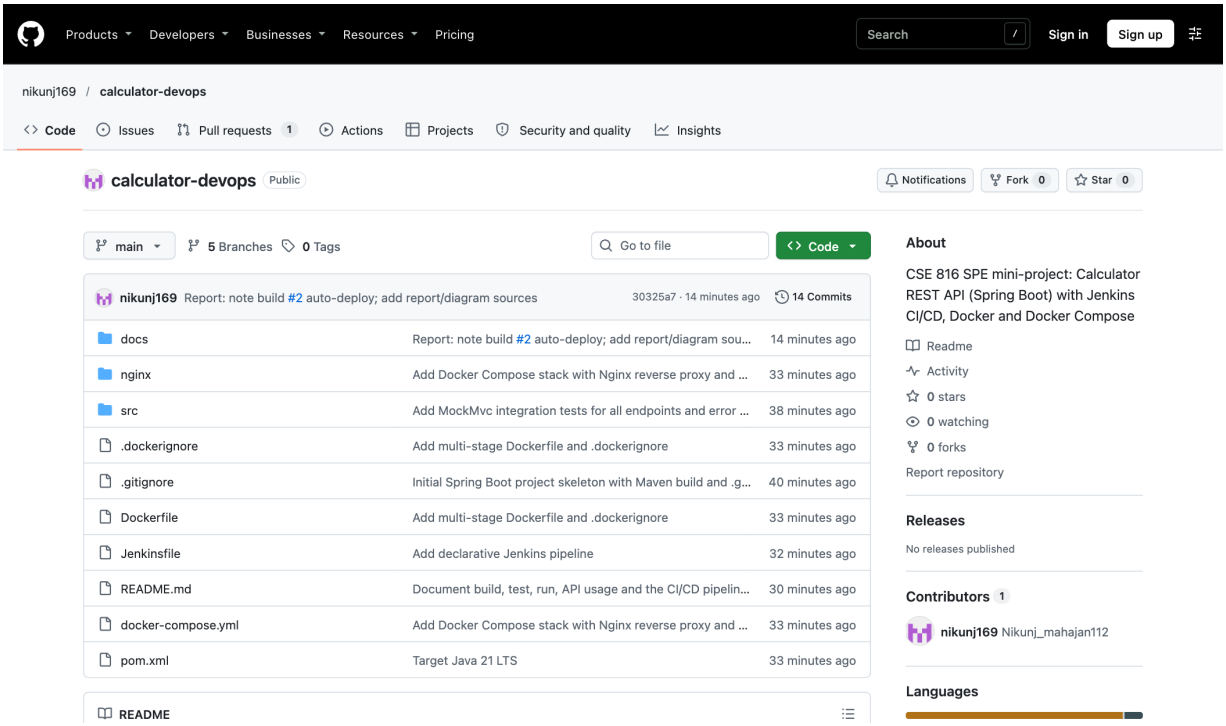


Figure 14 - The public GitHub repository.